

A Cyclic Calculus of Inductive Constructions: Escaping the Bureaucracy of Inductive/Recursive Syntax

Dr. Gavin Mendel-Gleason
Scidonia Limited
`scidonia.ai`

February 13, 2026

Abstract

This note motivates a cyclic variant of a Calculus of Inductive Constructions (CIC)-like calculus in which proofs are finite graphs with cycles, validated by a global progress condition, rather than trees generated by a fixed family of induction principles. This representation supports normalization-style transformations (commuting conversions, unfolding/refolding, etc.) that can *identify* proofs that are distinct syntactically, yielding a form of post-hoc proof identity. We outline (i) the source term calculus, (ii) a fix-free cyclic proof/term language with explicit substitution evidence, (iii) the global condition on cycles, and (iv) the use of *supercompilation* as a normalization procedure.

Contents

1	Introduction	2
2	Prior work and relation to Cyclic CIC	4
3	Why go cyclic? Proof identity and non-fixed induction principles	4
3.1	Example 1: cyclic proofs as proof graphs	5
3.2	Example 2: dependent index bureaucracy	5
3.3	The core motivation: induction principles need not be fixed	6
3.4	Post-transformation proof identity	7
4	Source calculus (terms, typing, semantics)	7
4.1	Term syntax	7
4.2	Typing	7
4.3	Call-by-name operational semantics	9
5	Cyclic CIC objects (proof graphs and fix-free vertex language)	9
5.1	Preproofs: locally-correct finite proof graphs	9
5.2	Cyclic proofs: global condition + witness	10
5.3	Semantics and intended equality	10
5.4	Fix-free vertex language (used by read-off)	10
5.5	Judgements over vertices	11

6	A cyclic sequent calculus for our CIC fragment (sketch)	11
6.1	Judgements	12
6.2	Core rules (Pi, lambda, application)	12
6.3	Inductives: roll and case (sketch)	12
6.4	Cyclic sequent proofs	12
7	Global conditions on cycles	13
7.1	Ranking condition (template)	13
7.2	Why this matters for transformations	14
8	Supercompilation for normalisation	14
8.1	Supercompilation (what it consists of)	14
8.2	Algorithm sketch (driving + whistle + generalise + fold)	15
8.3	Worked example: indexed types and cast elimination	15
8.4	Beta-reduction strategies	16
8.5	CASECASE: commuting nested case	17
8.6	Worked example: unfold/generalise/fold (cycle creation)	17
8.7	Information propagation in case scrutinee	18
8.8	Full normalization: driving under binders	19
8.9	Unfolding and refolding of cycles	19
9	Key theorems (mechanized)	20
9.1	Typed CIU equivalence	20
9.2	Head beta reduction	20
9.3	Read-off and extraction round-trip	20
9.4	Case-case commuting conversion	21
9.5	Lifting transformations	21
9.6	Ranking-based global progress (proved)	21
10	Canonical forms via supercompilation-style control	21
10.1	Control relation (homeomorphic embedding)	22
10.2	Reductions “modulo unfolding”	22
10.3	Canonical form (intended)	22
10.4	Connection to transformations	22
11	Conclusion	22

1 Introduction

In conventional systems, recursive reasoning is packaged as a fixed-point operator or as a family of induction/recursion schemata. Those choices are powerful, but they also hard-code a particular presentation of recursion into the syntax of proofs. As a result, proofs that are semantically equivalent can remain syntactically far apart, even after many semantics-preserving transformations.

Cyclic proof theory suggests a different decomposition:

A proof is a finite graph of locally-correct steps; recursion is represented by cycles; and soundness is ensured by a global progress condition.

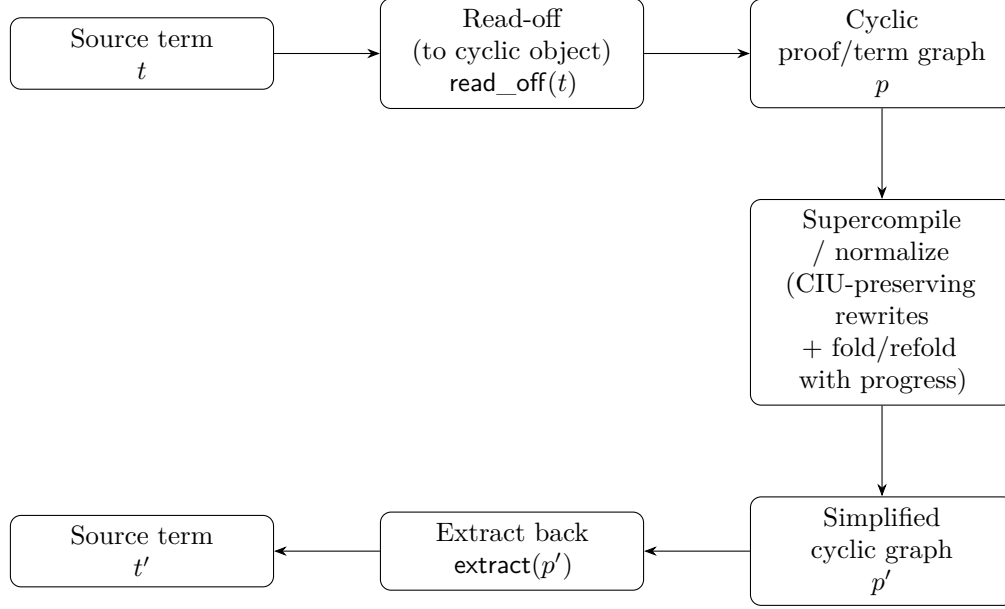


Figure 1: Normalization pipeline: compile terms to cyclic objects, simplify by supercompilation, then extract back to term syntax. The intended equality notion is $t \approx t'$ up to typed CIU, and fold/refold steps are constrained by global progress evidence.

This decomposition has an attractive consequence for proof engineering and normalization: we can refactor recursive structure (fold/refold), commute conversions, and perform administrative normalization without being forced to preserve the exact boundaries of `fix` binders or induction schemata. In the best case, normalization can reveal that two proofs are not just equivalent, but *the same cyclic object* after refolding.

Scope and claims. This paper specifies a cyclic CIC-like calculus, its extracted-term semantics, and a normalization pipeline based on supercompilation. Figure 1 summarizes the intended end-to-end workflow.

The canonicalisation claim is deliberately *operational*. We can guarantee that the partial-evaluation component is complete for the chosen head-reduction strategy (i.e. driving to weak-head normal form does not miss head redexes). For the full supercompilation bundle (driving + information propagation + commuting conversions + unfold/generalise/fold), a natural and desirable property is *idempotence* (a second pass yields no further change); at present we treat this as a conjecture/engineering target rather than a proved meta-theorem. However, we do *not* claim that all semantically identical programs/proofs can always be reduced to a single unique normal form. Rather, the goal is that many bureaucratic differences—especially those arising from commuting conversions, index computations, and recursion-boundary choices—are eliminated by the procedure.

The rest of the paper outlines:

- the source CIC-like term calculus (Section 4);
- the cyclic proof/term graph architecture (Section 5);
- the global condition on cycles (Section 7);
- and representative transformations like CASECASE and head beta-reduction (Section 8).

2 Prior work and relation to Cyclic CIC

This section positions the present development relative to (i) cyclic proof theory, (ii) proof-theoretic treatments of induction via global discharge conditions, and (iii) CIC kernel engineering.

Cyclic proof theory: from concrete systems to abstract frameworks. Cyclic proof systems represent potentially infinite derivations using finite graphs with back-edges, validated by a global soundness condition. This approach originates in cyclic proofs for inductive definitions and infinite descent [3, 4]. Afshari and Wehr develop a modern, highly abstract account of cyclic proof checking based on trace conditions, activation algebras, and categorical structure [1]. Our work is complementary in two ways: (1) we instantiate cyclic proofs inside a typed setting (a CIC-like calculus with evaluation and CIU (*closed instantiations of uses*) observational equivalence); (2) we focus on using cyclic proofs as *normal forms stable under program/proof transformations* (commuting conversions, unfold/refold, etc.). In particular, many of our proofs of interest are about preservation of observational equivalence under transformations rather than about the abstract complexity of proof checking.

Local induction vs global discharge. Sprenger and Dam compare a proof system with a local well-founded induction rule to a cyclic system validated by a global induction discharge condition, and give effective translations between the two (in the setting of the first-order μ -calculus) [8]. We use this as direct prior evidence for the core design thesis of this project: *the induction principle need not be fixed syntactically*. Where Sprenger–Dam study the relationship between local and global presentations of induction in a fixed-point logic, we pursue the same decomposition in a dependent type-theoretic setting, and then exploit it operationally: cycles become the unit of fold/refold during normalization and compilation of proof objects.

Engineering CIC kernels. Asperti et al. give a detailed kernel-level account of CIC checking in the Matita prover, emphasizing compactness and clear engineering boundaries [2]. Our contribution is not a new kernel design; instead, we propose a proof-object language where recursive structure is externalized as graph structure plus a global condition. Nevertheless, the kernel perspective is relevant: if cyclic proofs are to support a strong notion of post-transformation proof identity, then proof checking and definitional equality need representations that survive aggressive normalization and refolding.

Supercompilation and termination control. Supercompilation is a semantics-driven program transformation methodology centered on symbolic evaluation (“driving”), generalization, and fold/refold steps [9, 7]. A key practical ingredient is a *control* (or “whistle”) mechanism preventing infinite unfolding, often based on homeomorphic embedding and Kruskal-style well-quasi-ordering arguments [5, 6]. Our stance is that cyclic proof normalization is *precisely supercompilation*—but performed at the level of typed judgements/proof objects rather than untyped programs. The global cycle condition and the control relation play the same role they do in supercompilation: they justify refolding and prevent uncontrolled infinite unfolding.

3 Why go cyclic? Proof identity and non-fixed induction principles

We begin with tiny examples that make the motivation concrete: in ordinary CIC developments, the objects we want to treat as “the same proof” or “the same program” are often separated by

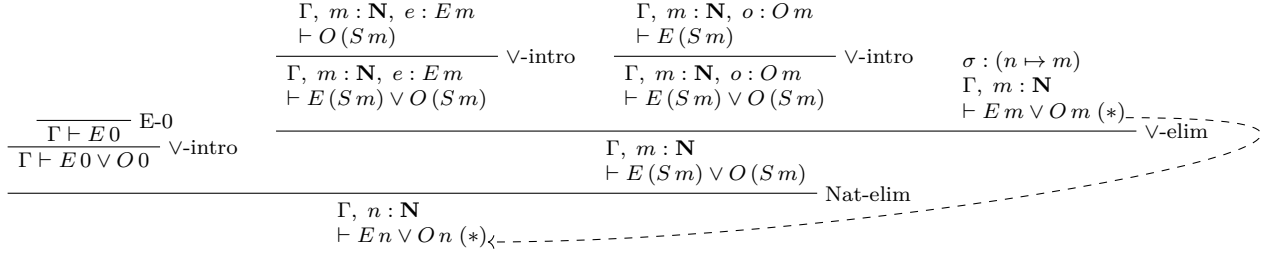


Figure 2: A schematic cyclic natural-deduction proof in CIC style (example shape due to [3]). Write En and On for evenness/oddness judgements over $\mathbf{N}$. We treat $A \vee B$ as an inductive type; proof terms are elided. The bud $(*)$ is discharged by a back-link to its companion, carrying explicit substitution evidence $\sigma : (n \mapsto m)$.

```

Definition pipeline_vec_raw (l : list A)
  : Vector.t C (length (map g (map f l))) :=
  Vector.of_list (map g (map f l)).

```

```

Definition pipeline_vec (l : list A) : Vector.t C (length l) :=
  eq_rect _ (fun n => Vector.t C n) (pipeline_vec_raw l)
  _ (length_map_map l).

```

Figure 3: Index bureaucracy caused by list pipeline composition: the index $\text{length}(\text{map } g(\text{map } f \ l))$ must be transported to $\text{length } l$ using an explicit equality proof.

administrative syntax. Cyclic proof objects aim to eliminate this bureaucracy by making recursion a matter of *graph structure* (cycles + explicit substitutions), leaving local typing and computation unchanged.

3.1 Example 1: cyclic proofs as proof graphs

Figure 2 shows the shape of a cyclic proof as a finite natural-deduction derivation with a back-link from a bud to its companion, carrying explicit substitution evidence.

Definition 3.1 (Cyclic proof (after [3], informal)). *A cyclic preproof is a finite proof graph whose nodes are labelled by judgements and locally justified by inference rule instances; some leaves (buds) are discharged by back-links to earlier occurrences of the same obligation (their companions). In our setting, each back-link additionally carries explicit substitution evidence describing how the bud is instantiated from its companion.*

A cyclic proof is a cyclic preproof equipped with a global trace/progress condition ensuring that the infinite unfolding obtained by repeatedly following back-links is sound (Section 7).

3.2 Example 2: dependent index bureaucracy

In dependently typed developments, index expressions are themselves programs. Even when these programs are total and terminating, definitional equality may not normalise them using fusion/algebraic laws, forcing explicit transports/casts. A representative pattern is:

In our mechanisation, we make this slogan concrete by running our supercompiler on such index programs and checking that the residualised indices coincide by computation (in Coq, by

`vm_compute`). Representative checked examples include:

- `residualisation_length_map_exact`
supercompiling `length(map f l)` yields exactly `length l`.
- `residualisation_length_append_plus_exact`
supercompiling `length(append l1 l2)` yields exactly `plusL(length l1)(length l2)`.
- `residualisation_length_append_assoc_fp_exact`
after iterating residualisation to a bounded fixed point, the two indices

$$\text{length}(\text{append}(\text{append } l_1 \ l_2) \ l_3) \quad \text{and} \quad \text{length}(\text{append } l_1(\text{append } l_2 \ l_3))$$

residualise to the same term.

We also package these index equalities as vector-index equalities; see `residual_vec_index_append_plus_exact` and `residual_vec_index_append_assoc_fp_exact`.

These examples motivate the rest of the paper: cyclic proofs aim to support semantics-preserving transformations (supercompilation-style driving plus fold/refold under a termination control) that eliminate administrative structure and yield canonical cyclic representatives.

A standard way to represent recursive reasoning in type theory is to bake recursion into the term language:

- either directly, via a fixed-point constructor (`fix`), or
- indirectly, via a pre-chosen set of induction/recursion schemata tied to the type formers.

This choice has a subtle but important consequence: it commits the system to a *particular syntactic presentation* of recursive reasoning. Two proofs may establish the same semantic property, but remain permanently different pieces of syntax because they encode recursion using different unfoldings, different induction schemata, or different administrative structure.

Of course, there cannot be a single fully automatic principle that always produces a canonical representative for *all* proofs and programs of interest. However, in many useful cases one can normalize aggressively enough (using a supercompilation-style bundle of semantics-preserving transformations, plus a termination control) that distinct presentations converge to the same cyclic normal form (up to a chosen graph equivalence). This is precisely where the approach pays off: it avoids proof bureaucracy in the common situation where the remaining differences are administrative rather than semantic.

3.3 The core motivation: induction principles need not be fixed

Cyclic proof theory takes a different stance:

The recursive structure of a proof is not a primitive binder or a fixed schema; it is a *cycle* in a finite proof graph.

A cycle does not, by itself, guarantee soundness. Instead we equip the finite graph with a *global progress condition* (Section 7) that rules out “bad” cycles and validates the intended infinite unfolding.

The key conceptual gain is *modularity of recursion*: we can change the cyclic structure (by folding, refolding, or commuting constructors through eliminators) without changing the local steps of the proof. In a fixed-point calculus, the recursion principle is pinned to the syntax of `fix`; in a schema-based setting it is pinned to the chosen eliminators. In a cyclic calculus, the recursion principle is an emergent property of the graph plus the global condition.

3.4 Post-transformation proof identity

This flexibility is not merely aesthetic. It enables a strong normalisation/canonicalisation story for proofs:

- Start from two proofs that are syntactically different (for example: one uses an unfolding step early, another delays it; one distributes a case earlier; one uses a different cycle boundary).
- Apply a sequence of semantics-preserving transformations (Section 8).
- After normalization/refolding, the resulting cyclic proof graphs may become *isomorphic* or even *literally identical* as finite objects.

In other words, transformations can expose that two proofs are “the same proof” once recursion is treated as *structure* (cycles) rather than as a fixed syntactic operator or schema. This is precisely the kind of identification that is difficult in traditional settings: the syntactic boundary of a **fix** binder (or the syntactic choice of induction schema) tends to be rigid, so normalization can preserve semantics while still leaving behind irreducible syntactic differences.

Cyclic proofs aim to make such equivalences more intensional: if two proofs normalize to the same cyclic representative (up to a chosen notion of graph equivalence such as bisimulation), we can treat them as equal in a strong, structurally meaningful sense.

4 Source calculus (terms, typing, semantics)

This section summarizes the *source* term language and its typing and operational semantics. It corresponds closely to the mechanized development (term syntax, typing, and call-by-name semantics).

4.1 Term syntax

We write terms t, u and types A, B using the following constructors (de Bruijn indices for variables):

$$t ::= \text{Var } x \mid \text{Sort } i \mid \Pi(A). B \mid \lambda(A). t \mid t u \mid \text{fix}(A). t \\ \mid \text{Ind}^I(\bar{a}) \mid \text{roll}_c^I(\bar{a}) \mid \text{case}^I(t; C; \bar{br})$$

Indexed inductive families. The system supports CIC-style indexed inductive families. An inductive $\text{Ind}^I(\bar{a})$ is applied to arguments $\bar{a}$ (parameters and indices). Constructors $\text{roll}_c^I(\bar{a})$ take all arguments (non-recursive and recursive) uniformly as $\bar{a}$. The dependent case eliminator $\text{case}^I(t; C; \bar{br})$ has a *dependent motive* C binding the scrutinee: under the binder, C has access to $x : \text{Ind}^I(\bar{id}x)$ where $\bar{id}x$ are the scrutinee’s indices.

4.2 Typing

A typing environment Σ stores inductive signatures (strictly-positive definitions). A context Γ is a list of types. The main judgement is $\Sigma; \Gamma \vdash t : A$.

Below is the core typing system (informally mirroring `Typing.has_type`). We omit the full inductive signature side-conditions and arity bookkeeping, but include the essential shape.

$$\frac{\Gamma(x) = A}{\Sigma; \Gamma \vdash \text{Var } x : A} \text{ (TyVar)}$$

$$\frac{}{\Sigma; \Gamma \vdash \text{Sort } i : \text{Sort } i + 1} \text{ (TySort)}$$

$$\frac{\Sigma; \Gamma \vdash A : \text{Sort } i \quad \Sigma; A :: \Gamma \vdash B : \text{Sort } j}{\Sigma; \Gamma \vdash \Pi(A). B : \text{Sort } \max(i, j)} \text{ (TyPI)}$$

$$\frac{\Sigma; \Gamma \vdash A : \text{Sort } i \quad \Sigma; A :: \Gamma \vdash t : B}{\Sigma; \Gamma \vdash \lambda(A). t : \Pi(A). B} \text{ (TyLAM)}$$

$$\frac{\Sigma; \Gamma \vdash t : \Pi(A). B \quad \Sigma; \Gamma \vdash u : A}{\Sigma; \Gamma \vdash t u : B[u/0]} \text{ (TyAPP)}$$

$$\frac{\Sigma; \Gamma \vdash A : \text{Sort } i \quad \Sigma; A :: \Gamma \vdash t : A \uparrow}{\Sigma; \Gamma \vdash \text{fix}(A). t : A} \text{ (TyFix)}$$

Typing indexed inductives. Let $\Sigma(I) = (\overline{P}, \overline{X}, \ell, \overline{ctor})$ where:

- $\overline{P}$ are uniform parameters
- $\overline{X}$ are index types
- ℓ is the universe level
- $\overline{ctor}$ are constructor specifications

Each constructor c has:

- Non-recursive argument types $\overline{\tau}_{param}$
- Recursive argument specifications (indices at which recursion occurs)
- Result indices $\overline{id}x_c$

Typing rules (informal):

- $\text{Ind}^I(\overline{a})$ has type $\text{Sort } \ell$ if $\overline{a}$ matches $\overline{P} ++ \overline{X}$
- $\text{roll}_c^I(\overline{a})$ has type $\text{Ind}^I((\overline{p} ++ \overline{id}x_c[\overline{a}]))$ where $\overline{p}$ are the parameter values
- $\text{case}^I(t; C; \overline{br})$:
 - Scrutinee: $\Sigma; \Gamma \vdash t : \text{Ind}^I(\overline{scrutIdx})$
 - Motive: $\Sigma; \Gamma, x : \text{Ind}^I(\overline{scrutIdx}) \vdash C : \text{Sort } i$ (dependent on scrutinee)
 - Each branch: $\Sigma; \Gamma \vdash br_c : \Pi \overline{\tau}_c. C[\text{roll}_c^I(\overline{a})/x]$
 - Result type: $C[t/x]$

(See the mechanized development for the full signature representation and typing rules.)

4.3 Call-by-name operational semantics

We use a weak-head, call-by-name reduction relation $t \rightarrow u$ (see `Semantics/Cbn.v`).

Key rules include:

$$\frac{}{(\lambda(A).t) u \rightarrow t[u/0]} \text{ (STEP-BETA)}$$

$$\frac{t \rightarrow t'}{t u \rightarrow t' u} \text{ (STEP-APP1)}$$

$$\frac{}{\text{fix}(A).t \rightarrow t[\text{fix}(A).t/0]} \text{ (STEP-FIX)}$$

$$\frac{t \rightarrow t'}{\text{case}^I(t; C; \overline{br}) \rightarrow \text{case}^I(t'; C; \overline{br})} \text{ (STEP-CASESCRUT)}$$

$$\frac{\text{branch}(\overline{br}, c) = br}{\text{case}^I((\text{roll}_c^I(\overline{a})); C; \overline{br}) \rightarrow (br \ \overline{a})} \text{ (STEP-CASEROLL)}$$

In the STEP-CASEROLL rule, the runtime step selects the branch and applies it to the constructor arguments. Dependence is tracked by typing: the case expression has type $C[t/x]$ (i.e. the motive instantiated with the scrutinee), even though the motive does not appear at runtime.

We write $t \rightarrow^* u$ for the reflexive-transitive closure of $\rightarrow$. Values are λ -abstractions and constructor rolls.

5 Cyclic CIC objects (proof graphs and fix-free vertex language)

There are two related layers in the development:

- (1) a generic notion of *preproof* / *cyclic proof* as a finite digraph whose nodes are labelled by judgements;
- (2) a concrete *fix-free* vertex language (nodes like `nLam`, `nApp`, `nBack`) used by read-off/extraction.

5.1 Preproofs: locally-correct finite proof graphs

A preproof is a finite directed graph together with:

- a node-label function $\text{label} : V \rightarrow \text{Judgement}$
- a successor function $\text{succ} : V \rightarrow \text{list } V$
- a local correctness property stating that each node is justified by a rule instance.

Concretely, we can package this as a function `Rule` that checks a node label against the labels of its successors:

$$\text{local_ok}(v) \stackrel{\text{def}}{=} \text{Rule}(\text{label}(v), \text{map label } (\text{succ}(v)))$$

This corresponds to `theories/Preproof/Preproof.v`. A *rooted* preproof additionally designates a root vertex.

Buds, back-links, and explicit substitutions. To represent recursion explicitly, we distinguish a subset of vertices $B \subseteq V$ (*buds*) whose outgoing edge is a *back-link* to an earlier vertex (their *companion*). Concretely, we record:

- a companion function $\text{comp} : B \rightarrow V$, and
- substitution evidence $\text{sv} : B \rightarrow V$ (a vertex encoding a list-backed explicit substitution).

Intuitively, a bud judgement is an instance of its companion judgement, and $\text{sv}(b)$ witnesses the instantiation. This is the key extra structure beyond an ordinary locally-correct proof tree.

5.2 Cyclic proofs: global condition + witness

A cyclic proof packages a preproof together with an additional witness W and a predicate `progress_ok` establishing the global soundness condition. The role of W is to justify that every directed cycle contains progress (e.g. via a well-founded ranking or trace argument), preventing unsound self-justifying loops. See `theories/CyclicProof/CyclicProof.v`.

5.3 Semantics and intended equality

For the purposes of this paper, we take the meaning of a cyclic object to be given by its *extracted term*. That is, a cyclic proof/term graph p denotes the source-term $\text{extract}(p)$ produced by the read-off/extraction pipeline (Section 9).

Accordingly, the intended equality of cyclic objects is not raw graph isomorphism but *semantic equality of extracted terms*: we regard p and q as representing the same proof/program when their extracted terms are equivalent up to typed CIU:

$$p \approx q \stackrel{\text{def}}{\iff} \text{ciu_jTy}(\Sigma, \Gamma, \text{extract}(p), \text{extract}(q), A)$$

for the relevant typing judgement. This choice is deliberate: it lets us talk about canonical representatives in a way that matches the mechanized observational semantics, and it avoids committing to a particular quotient on finite graphs before the refolding theory is finalized.

5.4 Fix-free vertex language (used by read-off)

The read-off compiler produces a fix-free cyclic term graph whose node labels are:

```
nVar x | nSort i | nPi | nLam | nApp
nInd I | nRoll (I, c, np, nr) | nCase (I, nbr)
nSubstNil k | nSubstCons k | nBack
```

Key design points (see `theories/Transform/ReadOff.v`):

- There is *no* fix node label.
- Recursion is represented by `nBack` nodes, which point to: (i) a cycle-target vertex, and (ii) an explicit substitution-evidence vertex.
- Substitutions are represented as linked lists of evidence nodes: `nSubstNil` and `nSubstCons`, mirroring list-backed substitutions.

Reintroducing `fix` when extracting back. Although the cyclic intermediate representation is `fix`-free, when we reify it back into an ordinary source term we may reintroduce `fix` to represent back-links. Intuitively, a back-link becomes a recursive call to a named fixed point, and the stored substitution evidence becomes the explicit instantiation of its parameters.

This is precisely why the cyclic IR increases identification power: two source terms that differ only in where recursion is packaged (different `fix` boundaries, different unfoldings, different administrative reshufflings) can still compile to cyclic graphs that fold/refold to the same canonical structure, and hence have extracted terms equal up to typed CIU.

5.5 Judgements over vertices

A first-cut judgement language over vertices (see `theories/Transform/CyclicRules.v`) is:

$$\text{jTy } \Gamma \ v \ A \quad | \quad \text{jEq } \Gamma \ v \ w \ A \quad | \quad \text{jSub } \Delta \ sv \ \Gamma$$

Intuitively:

- `jTy  $\Gamma \ v \ A$` : vertex v has type-vertex A under context of vertices Γ .
- `jSub  $\Delta \ sv \ \Gamma$` : substitution vertex sv is well-typed as a substitution from Γ into Δ .
- `jEq` provides intensional equality structure (initially `refl/symm/trans`; computation rules are planned).

The rules reference abstract graph-level operations `shiftV` and `substV`, intended to be implemented by `fix`-free rewriting (or by derived operations on the graph).

Backlink rule (core idea). A backlink node `nBack` represents a recursive call as an instantiation of an earlier obligation:

$$\frac{\text{jTy } \Gamma_0 \ target \ A_0 \quad \text{jSub } \Gamma \ sv \ \Gamma_0}{\text{jTy } \Gamma \ back \ (\text{substV } sv \ A_0)} \text{ (TyBack)}$$

This is where recursion lives in the cyclic object: the graph edge from *back* to *target* closes the cycle, while sv records how the recursive call is instantiated.

6 A cyclic sequent calculus for our CIC fragment (sketch)

The cyclic calculus in this paper is presented in a natural-deduction style: proof terms are built from introductions and eliminations, and cyclic structure appears as back-links in a finite proof graph. If this turns out to be an awkward presentation for proof identity or normalization, a reasonable alternative is to present the same core ideas in a *sequent-calculus* style. This section is a design sketch of such a presentation for the same CIC-like term language used throughout the mechanisation.

Why a sequent calculus? A sequent calculus makes *structural rules* (weakening, exchange, contraction, cut) explicit and treats eliminations as left rules. This can make the interaction between (i) commuting conversions, (ii) explicit substitutions, and (iii) fold/refold steps feel less “baked into” term syntax. It can also be a more convenient starting point for graph-based proof objects, since proof graphs are naturally close to derivation DAGs.

6.1 Judgements

We write two mutually defined typing sequents, in the style of bidirectional typing:

$$\Sigma; \Gamma \vdash t \Rightarrow A \quad (\text{synthesis}) \qquad \Sigma; \Gamma \vdash t \Leftarrow A \quad (\text{checking})$$

Intuitively, synthesis sequents compute a type for a term, while checking sequents verify a term against a known type. Type formation sequents and universe levels are elided in this sketch; the intent is that they match the source calculus of Section 4.

6.2 Core rules (Pi, lambda, application)

The following rules are representative. We use the paper’s lightweight inference macro `\Infer` (see `macros.tex`).

$$\begin{array}{c} \overline{\Sigma; \Gamma, x : A \vdash x \Rightarrow A} \text{ (VAR)} \qquad \overline{\Sigma; \Gamma \vdash \lambda(x : A). t \Leftarrow \Pi(x : A). B} \text{ (LAM)} \\ \text{where } \Sigma; \Gamma, x : A \vdash t \Leftarrow B \\ \\ \overline{\Sigma; \Gamma \vdash t u \Rightarrow B[u/x]} \text{ (APP)} \\ \text{where } \Sigma; \Gamma \vdash t \Rightarrow \Pi(x : A). B \text{ and } \Sigma; \Gamma \vdash u \Leftarrow A \end{array}$$

Cut / explicit substitution. A sequent presentation typically treats substitution as a derived admissible operation, with *cut* as its proof-theoretic counterpart. To make sharing and cyclic structure explicit, one can either: (i) add an explicit-substitution term former (a typed `let`), or (ii) treat cut as a primitive rule in the cyclic proof graph. A term-assignment flavour of cut can be written as:

$$\frac{\Sigma; \Gamma \vdash t \Rightarrow A \quad \Sigma; \Gamma, x : A \vdash u \Rightarrow C}{\Sigma; \Gamma \vdash u[t/x] \Rightarrow C} \text{ (CUT)}$$

In the cyclic setting, this is one place where a back-link edge can naturally live: the bud is a cut-premise whose solution is provided by a previously constructed node (Section 7).

6.3 Inductives: roll and case (sketch)

We keep the same surface constructors as elsewhere: $\text{roll}_k^t(\vec{t})$ for constructor application and $\text{case}^t(s; \vec{p}; \vec{t})$ for case analysis. A sequent formulation does not change the operational content, but it encourages presenting **case** as the primary elimination/left rule for inductives.

$$\overline{\Sigma; \Gamma \vdash \text{roll}_k^t(\vec{t}) \Rightarrow \text{Ind}^t(I)} \text{ (ROLL)} \qquad \overline{\Sigma; \Gamma \vdash \text{case}^t(s; \vec{p}; \vec{t}) \Rightarrow C} \text{ (CASE)}$$

Informally, **ROLL** checks the constructor arguments against the constructor telescope, and **CASE** checks the branches against the motive instantiated by each constructor. A full account would spell out the constructor telescopes, the motive, and the branch checking judgement. The important point for this note is that **case** is the canonical place to attach information-propagation and commuting-conversion transformations.

6.4 Cyclic sequent proofs

A *cyclic sequent preproof* can be defined exactly as in the natural-deduction presentation, but with nodes labelled by sequents (either synthesis or checking). A back-link edge discharges a bud node by pointing to a companion node with the same goal up to instantiation.

Back-links with instantiation. In our setting, back-links carry explicit substitution evidence. For sequents this can be presented as a (typed) context morphism that instantiates the bud context into the companion context. Concretely, a back-link from a bud labelled $\Sigma; \Gamma \vdash t \Rightarrow A$ to a companion labelled $\Sigma; \Gamma' \vdash t' \Rightarrow A'$ would additionally carry evidence that Γ is obtained by instantiating Γ' , and that applying that instantiation to t' and A' yields t and A .

Global progress. Soundness again requires a global condition: every infinite unfolding of back-links must witness progress according to a chosen well-founded measure. The same trace-based progress condition of Section 7 can be reused unchanged; only the node labels (sequents rather than ND judgements) differ.

Open design questions. This sketch leaves several choices open:

- Whether to adopt focusing (e.g. separate “invertible” and “non-invertible” phases) to control proof search and make commuting conversions canonical.
- Whether to treat cut as primitive in the proof object, or to represent sharing/instantiation exclusively through explicit substitutions attached to back-links.
- How to present dependent pattern matching in a sequent calculus so that the computational behavior (iota reduction) aligns with the extraction semantics used in the rest of the paper.

The main takeaway is that a cyclic sequent calculus is a viable alternative presentation, and it may provide a cleaner substrate for proof-graph identity and normalization if the natural-deduction surface ends up being too rigid.

7 Global conditions on cycles

Local correctness is not enough: an arbitrary cyclic preproof may be unsound. Cyclic proof theory therefore adds a *global* condition to rule out “bad” cycles.

The repository currently develops a ranking-style template (see `theories/Progress/Ranking.v`).

7.1 Ranking condition (template)

Fix a finite digraph G with vertices V . Assume:

- a predicate `progress_edge`(v, w) selecting edges that must make strict progress,
- a ranking domain M with well-founded strict order $<_M$,
- a rank function $\text{rank} : V \rightarrow M$.

The ranking condition requires:

- Well-foundedness:** $<_M$ is well-founded.
- Monotonicity:** along every edge $v \rightarrow w$, ranks do not increase.
- Strictness on progress edges:** if $v \rightarrow w$ is a progress edge, then $\text{rank}(w) <_M \text{rank}(v)$.
- Every directed cycle has progress:** every directed cycle in G contains at least one progress edge.

7.2 Why this matters for transformations

Fold/refold invariants (informal checklist). Any fold/refold transformation on cyclic objects is required to preserve:

- **Local correctness:** each node remains justified by a rule instance.
- **Well-typed substitution evidence:** every back-link continues to carry a well-typed substitution witness instantiating the companion judgement.
- **Progress witness:** the transformed graph admits a progress witness (ranking/trace) validating all directed cycles.
- **Extracted-term meaning:** extraction/read-off commutes with the graph transform up to typed CIU.

These are the invariants that prevent unsound refolding: a fold is only accepted when it can be accompanied by progress evidence and typing of the substitution witness.

Transformations that introduce, remove, or restructure cycles (fold/refold, read-off compilation, or higher-level normalisation steps) are only meaningful if the resulting cyclic object still has a progress witness.

Conceptually:

Finite graph + local rule correctness + global cycle progress $\Rightarrow$ sound infinite unfolding/meaning.

Thus, semantic preservation results for transformations typically factor into:

- (1) a *local* preservation argument (the transformation preserves the intended judgement/evaluation meaning), and
- (2) a *global* argument transporting the progress witness (or rebuilding one).

8 Supercompilation for normalisation

This section outlines the main transformations currently being developed at the term level, with the intended lift to cyclic proof objects via extraction. The motivating theme is not merely that these rewrites preserve observational equivalence, but that they support a notion of *canonical cyclic representatives*.

8.1 Supercompilation (what it consists of)

In this development, “supercompilation” is used in the classical sense of semantics-driven normalization by a bundle of CIU-preserving transformations. Concretely, it consists of the following ingredients, applied iteratively under a termination control (a “whistle”):

1. **Partial evaluation** (driving/head reduction): evaluate known redexes (beta, fix-unfolding, and case-of-constructor) to expose computation.
2. **Information propagation:** push known information (especially from scrutinees) into motives/branches so later transformations can fire.

3. **Case distribution** (case commuting conversions): redistribute/delay case analysis by commuting nested cases (CASECASE).
4. **Unfolding**: expand definitions/cycles to expose new redexes (term-level fix unfolding, and graph-level unfolding of back-links/sharing).
5. **Generalisation**: when unfolding would diverge, replace a configuration by a more general one (guided by embedding/wqo arguments).
6. **Folding/refolding**: detect repeated (generalised) configurations and introduce sharing/back-links again.

The remainder of the section spells out the term-level instances of these steps and how they connect to the cyclic setting.

8.2 Algorithm sketch (driving + whistle + generalise + fold)

The intended normalization procedure is a standard supercompilation loop. We present a lightweight pseudo-code account; details of the termination control and generalisation policy are discussed in Section 10.

supercompile(t)

1. Initialize memo table $M := \emptyset$.
2. **drive**(t, M):
 - (a) Compute a weak-head normal form by repeated head steps (beta/fix/iota).
 - (b) Apply commuting conversions (e.g. CASECASE) and information propagation.
 - (c) If the whistle blows (embedding repeats), **generalise** the current configuration.
 - (d) If a (generalised) configuration matches a memoized one, **fold** by introducing a back-link with substitution evidence.
 - (e) Otherwise memoize it and recursively drive subterms (including under binders/motives).

This algorithmic view is the basis for our notion of canonicalisation: although no fully automatic procedure can canonise *all* terms, in many useful cases this loop normalises away administrative structure (commuting conversions, bureaucratic casts, and refactoring of recursion boundaries) while preserving typed CIU.

8.3 Worked example: indexed types and cast elimination

We return to the index-bureaucracy pattern from Figure 3. The essential computational fact is that mapping does not change length: $\text{length}(\text{map } h \ l) = \text{length}(l)$ and therefore $\text{length}(\text{map } g(\text{map } f \ l)) = \text{length}(l)$.

Two terms. Let $\text{Vec } A \ n$ be a length-indexed vector type. Assume we have a term

$$t_{\text{raw}}(l) : \text{Vec } C \ (\text{length}(\text{map } g(\text{map } f \ l))).$$

In an intensional kernel, to obtain a term at the more natural index $\text{length}(l)$, one often inserts an explicit transport (e.g. **eq_rect** in Coq):

$$t_{\text{cast}}(l) : \text{Vec } C \ (\text{length}(l)).$$

By contrast, an “ideal” normal form would be a term $t_{\text{nf}}(l)$ whose index is already definitionally $\text{length}(l)$, requiring no transport.

How supercompilation removes the bureaucratic index. To make the algorithmic content explicit, consider the subterm $\text{length}(\text{map } h \ l)$. After one unfolding step of **map** and **length** (partial evaluation), we obtain a nested case-of-case configuration. The CASECASE commuting conversion then distributes the outer case, yielding a single case on the scrutinee l . At that point, the algorithm has exposed that the recursive call pattern is identical to $\text{length}(l)$, and it can fold the configuration back into a shared cyclic representative.

In the dependent setting, information propagation is essential when the length computation appears inside a dependent motive (e.g. a motive producing $\text{Vec } C \ (\text{length}(\cdot))$). When the scrutinee is a constructor, motive propagation and scrutinee propagation (both proved CIU-preserving) make the instantiated motive explicit, so the index reduces along with the scrutinee.

End-to-end identity claim (up to CIU). Let $\text{NF}(t)$ denote the extracted term obtained by compiling to a cyclic object, supercompiling, and extracting back (Figure 1). Operationally, the above reasoning implies that $\text{NF}(t_{\text{cast}})$ and $\text{NF}(t_{\text{raw}})$ compute the same vector term with index $\text{length}(l)$, and hence that t_{cast} and $\text{NF}(t_{\text{cast}})$ are identified up to typed CIU.

Proof sketch (CIU, algorithm-driven). The proof follows the algorithm:

1. Each driving step (beta/fix/iota, CASECASE, motive propagation, scrutinee propagation) is justified by a proved typed-CIU theorem.
2. Each fold/refold step is justified by preservation of local typing and a transported progress witness.
3. Composing these steps yields $\text{ciu_jTy}(t, \text{NF}(t), A)$ for the specific terms in this example.

This is exactly the sense in which supercompilation establishes post-hoc identity of indexed terms: it normalises index computations and removes transports that were present only due to intensional definitional equality limitations.

8.4 Beta-reduction strategies

The mechanized development defines several reduction strategies:

Single-step head reduction.

$$\text{beta_reduce_once}(t) \triangleq \begin{cases} (\lambda(A). \text{body}) \ u \mapsto \text{body}[u/0] & \text{if } t = (\lambda(A). \text{body}) \ u \\ t & \text{otherwise.} \end{cases}$$

Weak-head normal form (WHNF). $\text{whnf_reduce}(n, t)$ iterates head reductions until reaching WHNF or exhausting fuel n . This reduces only at the head position, matching call-by-name semantics.

Full normalization. `full_normalize(n, t)` reduces everywhere, including under binders:

- First reduces to WHNF at the head
- Then recursively normalizes all subterms
- Descends under λ , Π , `fix`, and case motives

The development proves a CIU-style correctness theorem for head reduction:

$$\Sigma; \Gamma \vdash t : A \implies \text{ciu_jTy } \Sigma \Gamma t (\text{beta_reduce_once}(t)) A.$$

8.5 CaseCase: commuting nested case

The file `theories/Transform/CaseCase.v` implements a commuting conversion for nested cases.

Canonicalisation example (term-level). Consider two terms that differ only by administrative placement of case analysis:

$$t_1 \triangleq \text{case}^J((\text{case}^I(x; C; \overline{br_I}); D; \overline{br_J}); \text{and} \quad t_2 \triangleq \text{case}^I(x; C; \overline{br'_I}).$$

The CASECASE transformation deterministically rewrites t_1 to t_2 by distributing the outer case into each inner branch. Under our notion of cyclic-object equality (same extracted term up to typed CIU, Section 5), this means t_1 and t_2 share the same canonical representative after one driving step:

$$\text{ciu_jTy}(\Sigma, \Gamma, t_1, t_2, A).$$

This is a minimal but important instance of “bureaucracy elimination”: two syntactic presentations of the same computation/induction structure are identified by a CIU-preserving commuting conversion.

Informally:

$$\text{case}^J((\text{case}^I(x; C; \overline{br_I}); D; \overline{br_J}); \rightsquigarrow \text{case}^I(x; C; \overline{br'_I})$$

where each inner branch is transformed to:

$$\overline{br'_I} \triangleq (\text{rebuild a lambda prefix for the constructor arity}) \dots \text{ then } \text{case}^J((\overline{br_I} \ \bar{a}); D; \overline{br_J}).$$

In the development, the constructor arity/types are obtained from the inductive signature environment, and the branch transform is implemented by a function `commute_branch_typed_rec`.

8.6 Worked example: unfold/generalise/fold (cycle creation)

A key difference between ordinary rewriting and supercompilation is the ability to *create and re-use cycles*. Schematic example: consider a structurally recursive function written with explicit `fix`, and the same computation written by inlining one unfolding step. At the source term level these are distinct pieces of syntax; after extraction to a cyclic graph, both can refold to the same back-link structure.

Concretely, suppose we have a call-by-name term $t \triangleq \text{fix}(A). \lambda(n : \mathbb{N}). \text{case}^I(n; C; \overline{br})$ where one branch contains a recursive call. Supercompilation proceeds by:

1. **Unfolding** the `fix` once to expose the recursive call site.
2. **Driving** until it encounters a configuration already seen up to embedding.

3. **Generalising** (if necessary) to a more general call pattern.
4. **Folding** by introducing a back-link to the memoized configuration, with explicit substitution evidence recording how parameters are instantiated.

The result is a finite cyclic object whose extracted term is CIU-equivalent to the original, but whose recursion boundary is now canonicalized by fold/refold rather than fixed by the original `fix` placement. This is the intended mechanism by which the cyclic intermediate form can identify more terms: two terms with different unfold/fold structure can still map to cyclic objects with the same extracted semantics up to CIU.

8.7 Information propagation in case scrutinee

When the scrutinee of a case is known to be a constructor, we can propagate this information in two complementary ways:

Motive propagation. Instantiate the motive with the known constructor:

$$\text{case}^I((\text{roll}_c^I(\bar{a})); C; \overline{br}) \rightsquigarrow \text{case}^I((\text{roll}_c^I(\bar{a})); C[\text{roll}_c^I(\bar{a})/x]; \overline{br})$$

This is semantically a no-op at runtime (the motive is erased by reduction), but it makes the instantiated motive explicit and can enable further simplifications. The transformation is implemented as `propagate_motive_once` and proved CIU-preserving by `ciu_jTy_propagate_motive_once`.

Scrutinee propagation into branches. More aggressively, we can specialize the selected branch knowing the scrutinee’s constructor form. When the scrutinee is $\text{roll}_c^I(\bar{a})$, the branch br_c will receive arguments $\bar{a}$ at runtime. We can transform br_c by:

1. Computing the branch’s result type with the motive fully instantiated: $C[\text{roll}_c^I(\dots)/x]$
2. Reducing/simplifying this instantiated type (e.g. if C itself contains a case-of-constructor, it can reduce)
3. Adjusting the branch body to take advantage of the simplified result type

Formally:

$$\text{case}^I((\text{roll}_c^I(\bar{a})); C; \overline{br}) \rightsquigarrow \text{case}^I((\text{roll}_c^I(\bar{a})); C; [br_0, \dots, br'_c, \dots])$$

where br'_c is the specialized version of br_c .

Example. If the motive is $C = \lambda x. \text{case}^I(x; D; \dots)$, then knowing $x = \text{roll}_c^I(\bar{a})$ allows us to reduce the inner case in the result type, potentially unlocking further simplifications in the branch body.

This transformation is CIU-preserving because the branch’s observable behavior (which arguments it receives, what value it computes) is unchanged; we are only making the type information more concrete.

Implementation status: This transformation is implemented with a proved CIU preservation theorem in `theories/Transform/ScrutineeProp.v`.

8.8 Full normalization: driving under binders

For supercompilation-style transformations, we need to normalize terms fully, not just to weak-head normal form. This requires *driving under binders*: reducing subterms even when they appear under λ , Π , or case motives.

Driving strategy. The supercompiler (see `theories/Transform/Supercompile.v`) uses a fuel-limited driving function that applies the following transformations iteratively (cf. the supercompilation bundle above):

1. **Partial evaluation:** head reductions (beta, fix-unfolding, iota)
2. **Information propagation:** propagate known scrutinee into motive/branches
3. **Case distribution:** commute nested case expressions (CASECASE)
4. **Unfolding:** expand cyclic structure to expose more redexes
5. **Generalise and fold:** prevent divergence and re-introduce sharing/back-links
6. **Descend under binders:** recursively drive subterms under λ , Π , and case motives

Together these steps constitute the intended supercompilation-based normalization procedure for cyclic proofs.

Eta-expansion. A key question for full normalization is whether we need eta-rules:

$$\begin{array}{ll} \text{(eta-lambda)} & \lambda x. (t\ x) \equiv t \quad \text{if } x \notin FV(t) \\ \text{(eta-case)} & \text{case}^I(t; C; \overline{br}) \equiv t \quad \text{if each } br_c = \text{roll}_c^I(\dots) \end{array}$$

Current status: The implementation does *not* require eta-rules for correctness. Eta-expansion can be added as an optional normalization step if needed for canonical forms, but observational equivalence (CIU) is preserved without it.

8.9 Unfolding and refolding of cycles

Beyond term-level rewrites, cyclic proof objects admit transformations that restructure graph sharing and cycles:

- **Unfold:** expand a back-link or a shared subgraph, temporarily increasing size but exposing computation.
- **Fold/refold:** identify repeated sub-derivations and introduce a back-link, or rearrange cycle boundaries (e.g. SCC-normal forms).

These transformations are the point where cyclicity pays off. Because recursion is represented structurally (by cycles) rather than by a fixed syntactic operator, refolding can change *which* cyclic structure represents a proof without changing what it proves.

Why refolding is sound. Refolding is only sound when the resulting cycles satisfy a global progress condition. In our development, cyclic objects are packaged together with explicit progress evidence (Section 7); transformations that introduce or move back-links are required to transport or rebuild this evidence. Operationally, this blocks unsound refoldings that would validate an ill-founded “loop”: any directed cycle must contain a designated progress edge, and a well-founded ranking (or trace-style witness) must justify that the cycle makes genuine progress.

Thus, the normalization story factors cleanly: local CIU theorems justify term-level rewrites on extracted terms, while progress-preservation arguments justify the fold/refold steps on graphs.

9 Key theorems (mechanized)

This section states the main semantic preservation theorems mechanized in the Coq development. Our semantic equivalence is CIU-style observational equivalence at a typing judgement (values as observables).

TCB boundary and why CIU. We do *not* propose to change the kernel definitional equality of CIC. Instead, supercompilation is an *external* normalization procedure whose output is justified by proved meta-theorems. Typed CIU is the right notion here because our transformations are operational (call-by-name) and are intended to preserve program behavior observed by termination to values under all well-typed closing substitutions. Thus, the trusted computing base remains the usual type checker plus the metatheory proofs establishing CIU preservation of each transformation.

9.1 Typed CIU equivalence

Write $\text{ciu_jTy}(\Sigma, \Gamma, t, u, A)$ for typed CIU equivalence (see `theories/Equiv/CIUJudgement.v`). Intuitively, t and u are equivalent at type A if, for every well-typed closing substitution by values, they terminate to the same values.

9.2 Head beta reduction

Let β_1 be head call-by-name beta reduction (`theories/Transform/BetaReduce.v`).

Theorem 9.1 (Head beta preserves typed CIU). *For all Σ, Γ, t, A ,*

$$\Sigma; \Gamma \vdash t : A \implies \text{ciu_jTy}(\Sigma, \Gamma, t, \beta_1(t), A).$$

This corresponds to the proved Coq theorem `ciu_jTy_beta_reduce_once`.

9.3 Read-off and extraction round-trip

Let `read_off_raw` compile a source term to a fix-free cyclic graph, and let `extract_read_off` be the specialized extraction back to terms. The Coq development proves a strong round-trip equation.

Theorem 9.2 (Round-trip identity). *For every term t ,*

$$\text{extract_read_off}(t) = t.$$

An immediate corollary is CIU preservation:

Theorem 9.3 (Round-trip preserves typed CIU). *For all Σ, Γ, t, A ,*

$$\text{ciu_jTy}(\Sigma, \Gamma, t, \text{extract_read_off}(t), A).$$

These correspond to the proved Coq theorems:

- `extract_read_off_id`
- `extract_read_off_ciu`

9.4 Case-case commuting conversion

What is proved (and what is not). The theorems in this section are all of the form “typed CIU is preserved by a one-step transformation”. They do not (yet) imply that the full supercompilation loop terminates on all inputs or yields unique normal forms. Rather, they provide the semantic building blocks used in the algorithmic pipeline of Section 8.

Let $\text{CaseCase}(\Sigma, t)$ be the one-step commuting conversion for nested case.

Theorem 9.4 (CASECASE preserves typed CIU). *For all Σ, Γ, t, A ,*

$$\Sigma; \Gamma \vdash t : A \implies \text{ciu_jTy}(\Sigma, \Gamma, t, \text{CaseCase}(\Sigma, t), A).$$

This corresponds to the proved Coq theorem `ciu_jTy_commute_case_case_once`.

9.5 Lifting transformations

As a first “lifting” result, the repository contains a term-level wrapper that uses the above CIU theorem to justify the CaseCase transform on extracted cyclic objects.

Theorem 9.5 (CaseCase transform preserves typed CIU (by reduction to term theorem)). *For all Σ, Γ, p, A , if p is well-typed at A , then applying CASECASE to p preserves typed CIU.*

This corresponds to the proved Coq theorem `case_case_transform_preserves_equiv`.

9.6 Ranking-based global progress (proved)

The cyclic proof layer uses a global condition to validate cycles. One convenient instantiation uses a natural-number ranking that is monotone on all edges and strictly decreasing on designated progress edges.

Proposition 9.6 (Natural-number ranking implies progress condition). *If a finite proof graph admits a rank $\rho : V \rightarrow \mathbb{N}$ such that ranks never increase along edges and strictly decrease along progress edges, and every directed cycle contains a progress edge, then the global progress condition holds.*

This corresponds to `progress_ok_nat` in `theories/Progress/Measures.v`.

10 Canonical forms via supercompilation-style control

The long-term goal is a theory of *canonical cyclic proofs*: intensional representatives stable under semantics-preserving transformations. In this project, the intended normalization procedure is explicitly inspired by supercompilation [9, 7].

10.1 Control relation (homeomorphic embedding)

Supercompilation requires a termination control (a “whistle”) preventing infinite unfolding. The repository implements a homeomorphic embedding relation on terms and extends it to typed judgements (`theories/Progress/Embedding.v`). We write $j_1 \preceq j_2$ for judgement embedding.

The classical meta-theorem (by Kruskal-style arguments [5]) is that homeomorphic embedding is a well-quasi-order on finite terms, which implies that an infinite sequence must contain an embedding pair. Operationally, this justifies the control rule:

If a newly produced configuration embeds a previous one, stop unfolding and refold (generalize/backlink) instead.

10.2 Reductions “modulo unfolding”

Let $\rightarrow$ be the operational step relation on terms (Section 4). Let $\rightsquigarrow$ denote a single *unfolding* step at a cyclic back-link (or, in the source calculus, unfolding a `fix`). We treat $\rightsquigarrow$ as admissible but controlled: it is only permitted when it does not violate the control relation.

10.3 Canonical form (intended)

We say a cyclic object is in *canonical form* if:

- (1) (No further reductions) There is no reduction step $p \rightarrow p'$ available in the cyclic object.
- (2) (Unfolding is controlled) Any unfolding step $p \rightsquigarrow p_1$ that would enable further reductions $p_1 \rightarrow^+ p_2$ is forbidden by the control relation: along the would-be unfolding/reduction trace, some configuration embeds a previous one (a “whistle” blows), so the procedure must refold rather than continue unfolding.

This definition mirrors supercompilation practice: normal forms are not merely “stuck”; they are *maximally reduced subject to a termination control*. The presence of cycles (back-links) turns this into a canonicalization problem for graphs rather than trees: refolding chooses cycle targets and substitutions, and the global progress condition validates that the chosen cycles are productive/sound.

10.4 Connection to transformations

Under this view, transformations like head beta and CASECASE are the “driving” steps, while read-off/extract and fold/refold manipulate the sharing and cycles. The control relation (embedding on typed judgements) is the mechanism that prevents infinite unfolding and makes the canonical form finite.

11 Conclusion

This project is motivated by a proof-engineering problem that is easy to recognize in large dependently typed developments: *semantic computation is often blocked by bureaucratic syntax*. Even when two derivations represent the same underlying recursive reasoning, the boundaries of `fix` binders, the choice of induction schema, and the placement of administrative rewrites can force the proofs to remain syntactically distinct. Our claim is that cyclic proof objects provide a principled way to factor this bureaucracy out: recursion becomes *graph structure* (cycles plus explicit substitution evidence), while local steps remain ordinary CIC-style typing and computation.

Three concrete examples (drawn from the companion notes in `../cyclic-paper`) illustrate why this matters:

- **Cyclic even/odd reasoning as a proof graph.** A simple parity proof can be represented as a finite derivation with a back-link from a bud to its companion, carrying explicit substitution evidence (e.g. $\sigma : (n \mapsto m)$). The cyclic structure expresses the recursion directly, without committing to a particular `fix` term or induction schema.
- **Index bureaucracy from harmless program composition.** Composing traversals such as `map g (map f l)` produces indices like `length (map g (map f l))` that are propositionally equal to `length l` but not definitionally equal, forcing explicit transports/casts. We view cyclic normalization (supercompilation-style driving plus fold/refold) as a way to normalise such index computations without strengthening kernel definitional equality.
- **Fix-free recursion via back-links with substitution witnesses.** In the extracted cyclic vertex language, there is no `fix` node label. Instead, recursive calls are explicit `nBack` nodes that point to (i) a cycle target and (ii) a linked-list substitution witness. This makes recursion refactorable: fold/refold can change cycle boundaries without changing local typing.

Against this background, the main contributions are:

- **A cyclic CIC calculus.** We give (to our knowledge) the first development of a CIC-style setting where proof objects are finite graphs with cycles, rather than trees with a fixed recursion/induction discipline.
- **Cyclic normalization as supercompilation.** We make a direct connection between cyclic proof normalization and supercompilation: driving steps correspond to local computation/commuting conversions and information propagation, while fold/refold and back-links correspond to memoization, generalization, and controlled reuse, governed by a termination control (homeomorphic embedding).
- **Verified meta-theory and transformation correctness.** Substantial parts of the pipeline and its supporting theorems are verified in Coq, including key semantic preservation results and the read-off/extraction round-trip theorem.

Verified results and current status. The mechanized development currently includes:

- a proved CIU preservation theorem for head beta reduction (Theorem 9.1);
- a proved CIU preservation theorem for the CASECASE commuting conversion (Theorem 9.4);
- a proved round-trip identity theorem for read-off and extraction (Theorem 9.2) and its CIU corollary (Theorem 9.3);
- a proved ranking-based sufficient condition for the global progress predicate (Proposition 9.6).

Future work is to complete the remaining transformation theorems (including more of the supercompilation bundle), strengthen the canonical-form theory, and refine the notion of canonical cyclic representatives (graph equivalence and refolding invariants).

Future direction: cycle η -elimination for semantically constant cycles. A natural additional normalization step is an “ η -elimination” principle for cyclic structure. Intuitively, if a cyclic strongly connected component (SCC) is observationally *constant*—every way of exiting the SCC yields the same value—then the recursion carried by that SCC is bureaucratic and can be erased.

Motivational example. Consider an extracted cyclic graph whose root is a back-link that ultimately returns a fixed value (e.g. a branch that always produces 0 regardless of the recursive call). Such constant-recursion artifacts can arise during supercompilation when a configuration is memoized and later discovered to compute a value independent of its parameters. Contracting the SCC to that value would simplify the cyclic object and improve canonicalisation.

Soundness criterion. The global progress condition alone does *not* justify deleting cycles; it only ensures soundness of cyclic reasoning. To eliminate a cycle we would require semantic evidence, e.g. a typed CIU proof that the extracted term of the SCC root is equivalent to some value v . With extracted-term semantics and equality up to typed CIU, this becomes a proof-carrying transformation: replace the cyclic component by v when accompanied by a derivation of $\text{ciu_jTy}(t, v, A)$.

References

- [1] Bahareh Afshari and Dominik Wehr. Abstract cyclic proofs. *Mathematical Structures in Computer Science*, 34:552–577, 2024.
- [2] Andrea Asperti, Wilmer Ricciotti, Claudio Sacerdoti Coen, and Enrico Tassi. A compact kernel for the calculus of inductive constructions. *Sādhanā*, 34(1):71–144, 2009.
- [3] James Brotherston. Cyclic proofs for first-order logic with inductive definitions. In *TABLEAUX*, 2006.
- [4] James Brotherston and Alex Simpson. Sequent calculi for induction and infinite descent. In *LICS*, 2011.
- [5] Joseph B. Kruskal. Well-quasi-ordering, the tree theorem, and Vazsonyi’s conjecture. *Transactions of the American Mathematical Society*, 1960.
- [6] Michael Leuschel. Homeomorphic embedding for online termination of partial deduction. In *Logic-Based Program Synthesis and Transformation*, 2002.
- [7] Morten Heine Sørensen and Robert Glück. An introduction to supercompilation. In *Partial Evaluation and Semantics-Based Program Manipulation*, 1995. Often cited introductory survey; venue details vary by edition.
- [8] Christoph Sprenger and Mads Dam. On the structure of inductive reasoning: Circular and tree-shaped proofs in the μ -calculus. In *Foundations of Software Science and Computation Structures (FoSSaCS 2003)*, volume 2620 of *Lecture Notes in Computer Science*, pages 425–440. Springer, 2003.
- [9] Valentin F. Turchin. *The Concept of a Supercompiler*. Springer, 1986. Classic reference introducing supercompilation.